【转载】第二讲——白盒加密攻击方法的选择

转载 (https://www.qinless.com/category/reprint) · 2022-04-01 · 578 次浏览

文章目录[隐藏]

- 转载 19 / 25
- 说说在前面
- 一、前言
- 二、识别白盒加密
  - 2.1 确切特征
  - 2.2 辅助特征
- 三、从白盒加密中提取密钥
  - 3.1 密码学分析
  - 3.2 差分故障攻击（DFA）
  - 3.3 差分计算攻击 （DCA）
- 四、尾声

转载 (https://www.qinless.com/series/zhuanzai) 19 / 25

【转载】unidbg hook 使用大全 (https://www.qinless.com/670)

【转载】ida 基本使用 常用快捷键 (https://www.qinless.com/732)

【转载】Frida和调试器共存的问题 (https://www.qinless.com/937)

【转载】Charles抓包Android最佳流量转发APP (https://www.qinless.com/939)

【转载】安卓自定义代码调用系统函数并编译so记录 (https://www.qinless.com/1020)

【转载】jnitrace 手机卡死黑屏问题解决 (https://www.qinless.com/1169)

【转载】Unidbg SO 逆向入门实战教程一 OASIS (https://www.qinless.com/1173)

【转载】Unidbg SO 逆向入门实战教程二 calculateS (https://www.qinless.com/1177)

【转载】Unidbg SO 逆向入门实战教程三 V2-Sign (https://www.qinless.com/1181)

返回顶部

【转载】Unidbg SO 逆向入门实战教程四 mfw (https://www.qinless.com/1185)

【转载】Unidbg SO 逆向入门实战教程五 qxs (https://www.qinless.com/1188)

【转载】Unidbg SO 逆向入门实战教程六 s (https://www.qinless.com/1192)

【转载】Unidbg SO 逆向入门实战教程七 main (https://www.qinless.com/1194)

【转载】Unidbg SO 逆向入门实战教程八 文件读写 (https://www.qinless.com/1201)

【转载】Unidbg SO 逆向入门实战教程九 blackbox (https://www.qinless.com/1204)

【转载】Unidbg SO 逆向入门实战教程十 simpleSign (https://www.qinless.com/1209)

【转载】android app 加密参数分析研究 hash aes (https://www.qinless.com/1634)

【转载】第一讲——从黑盒攻击模型到白盒攻击模型 (https://www.qinless.com/1642)

【转载】第二讲——白盒加密攻击方法的选择

【转载】第三讲——差分故障攻击的原理 (https://www.qinless.com/1647)

【转载】第四讲——差分故障攻击的工具链 (https://www.qinless.com/1653)

【转载】第五讲——使用源码进行DFA攻击 (https://www.qinless.com/1655)

【转载】第六讲——使用Frida进行DFA攻击 (https://www.qinless.com/1657)

【转载】第七讲——使用Unidbg进行DFA攻击 (https://www.qinless.com/1659)

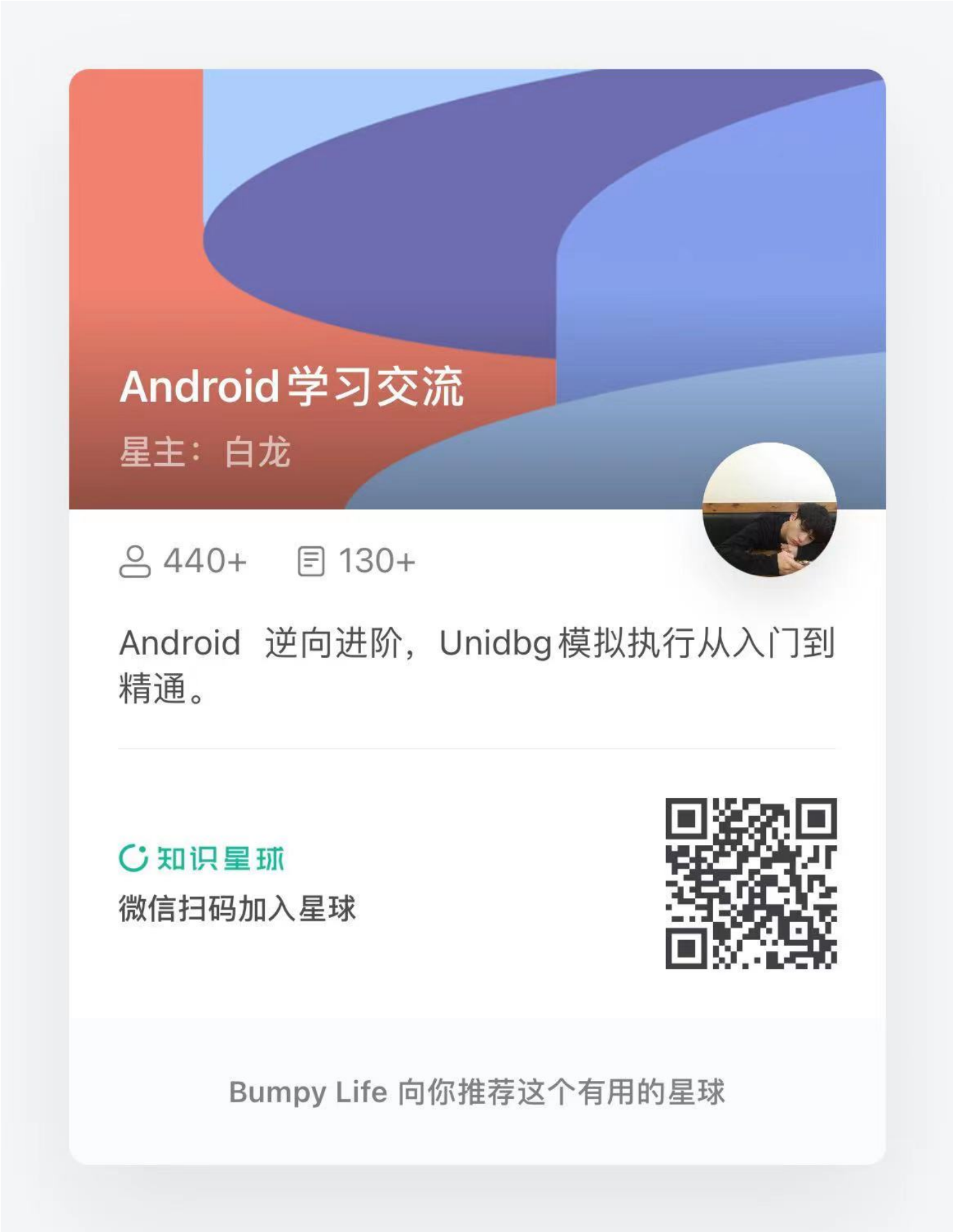
【转载】第八讲——案例 (https://www.qinless.com/1661)

« 上一篇文章 (https://www.qinless.com/1642)
下一篇文章 (https://www.qinless.com/1647) »

说说在前面

本文转载龙哥星球 白盒 AES 密码学系列，相关文件代码等，可加入星球下载

返回顶部



(https://www.qinless.com/wp-content/uploads/2022/03/longge-zsxq.jpeg)

返回顶部

一、前言

在这篇文章中，我们讨论从白盒加密中分析出密钥的技术。首先我需要声明，这个系列文章中并不关注白盒加密具体的实现，只关注于如何提取密钥。这听起来有点怪，都不了解加密算法如何白盒化，又怎么能做分析甚至是提取出密钥？

二、识别白盒加密

2.1 确切特征

白盒化一个加密算法的办法有很多，最常见的方案是查找表技术，我们只考虑基于查找表技术的这一类。查找表意味着将密钥和算法的其他步骤融进一张大表，将加密的过程转变成查表的过程。

那么它就会有两个特征

- 程序中使用到了一个或多个很大的数组，这个大指数组至少包含几千个元素。
- 原先轮函数中的运算都变成了查表

举个例子，如下是一份白盒AES原代码。

返回顶部

COPY

```
import ctypes

dword_6661C0 = []
byte_6651C0 = []
with open("tables/6661C0.txt", "r")as F:
    dword_6661C0 = eval(F.read().strip())
    F.close()

with open("tables/6651C0.txt", "r")as F:
    byte_6651C0 = eval(F.read().strip())
    F.close()

byte_6650C0 = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,1,0,3,2,5,4,7,6,9,8,11,10,13,12,15,14,2,3,

for i in range(len(dword_6661C0)):
    dword_6661C0[i]=ctypes.c_uint32(dword_6661C0[i]).value

for i in range(len(byte_6651C0)):
    byte_6651C0[i]=ctypes.c_uint8(byte_6651C0[i]).value

def swap(s):
    res = [i for i in range(16)]
    res[0] = s[0]
    res[1] = s[5]
    res[2] = s[10]
    res[3] = s[15]
    res[4] = s[4]
    res[5] = s[9]
    res[6] = s[0xe]
    res[7] = s[3]
    res[8] = s[8]
    res[9] = s[0xd]
    res[10] = s[2]
    res[11] = s[7]
    res[12] = s[0xc]
    res[13] = s[1]
    res[14] = s[6]
    res[15] = s[0xb]
    return "".join(res)

def xor(a,b):
    return "".join(chr(ord(a[i])^ord(b[i])) for i in range(16))
```

返回顶部

```
def encrypt(inp):
    for v8 in range(9):
        res = [i for i in range(16)]
        inp = swap(inp)
        for v9 in range(4):
            v3 = dword_6661C0[((4 * v9 + 16 * v8) << 8) + ord(inp[4*v9])]
            v4 = dword_6661C0[((4 * v9 + 1 + 16 * v8) << 8) + ord(inp[4*v9+1])]
            v5 = dword_6661C0[((4 * v9 + 2 + 16 * v8) << 8) + ord(inp[4*v9+2])]
            v6 = dword_6661C0[((4 * v9 + 3 + 16 * v8) << 8) + ord(inp[4*v9+3])]
            res[4*v9] = chr(byte_6650C0[(16*byte_6650C0[16*(v3 & 0xF)+(v4&0xf)]+byte_6650C0[16*
            v3 = v3 >> 8
            v4 = v4 >> 8
            v5 = v5 >> 8
            v6 = v6 >> 8
            res[4*v9+1] = chr(byte_6650C0[(16*byte_6650C0[16*(v3 & 0xF)+(v4&0xf)]+byte_6650C0[1
            v3 = v3 >> 8
            v4 = v4 >> 8
            v5 = v5 >> 8
            v6 = v6 >> 8
            res[4*v9+2] = chr(byte_6650C0[(16*byte_6650C0[16*(v3 & 0xF)+(v4&0xf)]+byte_6650C0[1
            v3 = v3 >> 8
            v4 = v4 >> 8
            v5 = v5 >> 8
            v6 = v6 >> 8
            res[4*v9+3] = chr(byte_6650C0[(16*byte_6650C0[16*(v3 & 0xF)+(v4&0xf)]+byte_6650C0[1
        inp = "".join(res)

    inp = swap(inp)
    res = [i for i in range(16)]

    for i in range(16):
        res[i] = "{:02x}".format(byte_6651C0[256*i + ord(inp[i])])
    inp = "".join(res)
    return inp

print(encrypt("0123456789abcdef"))
```

可以发现，主体运算变成了对dword_6661C0，byte_6650C0，byte_6651C0 这三个表的查表操作。其中dword_6661C0是如下的大表，其余两个是小表。

返回顶部

(https://www.qinless.com/wp-content/uploads/2022/04/6246feb2435b7.png)

白盒加密在SO中是否有其他特征？另一个比较好用的是符号和字符串信息，尽管听着有些不可思议，但我们经常可以看到一些SO叫 XXXWhiteBox，或者某个函数叫 Whitebox_init / WBAES / Whitebox_Sm4 这样的名字，这是非常明显的特征。

总结一下

- 逻辑上依赖"查大表"
- 函数或符号名涉及WhiteBox

需要注意，比如AES等算法存在表合并的实现，表合并和白盒化是两码事，表合并是将不涉及密钥的步骤合并在一起，白盒化是将密钥放进去。区分两者的主要方法是表合并的表是固定的，比如AES加密在表合并法后，就是OpenCL-AES (https://github.com/zliuva/OpenCL-AES/blob/master/eng_opencl_aes.cl)这个项目中的Te0-Te3四个表。

- 表合并法中的表固定，google就可以查到；白盒加密中的表查不到
- 白盒加密的表远比表合并中的更大，而且白盒加密中找不到Key的痕迹，表合并法AES必然找得到Key

在业界，也有很多人尝试从其他方面寻找白盒加密的特征，然后做成类似于 $FindCrypt$ 的 $FindWhitBoxCrypt$ ，但总体来说并不顺利，我们来看一下学者们给出的特征。

返回顶部

2.2 辅助特征

首先，研究人员发现，白盒密码的实现函数，往往是“巨大”且“独立”的。前者指的是，函数比样本中大多数函数更大，这个“大”包括：汇编的行数/基本块的数量/伪代码的长度等等，后者指白盒密码程序往往是一个单独的模块或独立的几个函数，而且不会使用太多库函数。这确实是很合理的推断，但一个标准加密，或者任意的、其他的关键逻辑，也都符合这种描述。

其次，研究人员认为，白盒函数所需要的栈空间往往较大，因为需要大量栈空间存储中间值。事实上，标准加密函数或混淆严重的普通函数也具有同样特征。

最后，比较有参考价值的一点是，如果一个函数在功能上十分像加密函数，比如AES，但是FIndCrypt 无法找到相关特征，基于对该种算法的了解也无法找到熟悉的逻辑，比如密钥编排，同时存在大表以及查表行为，那么可以往白盒加密的角度思考。

三、从白盒加密中提取密钥

如何提取出白盒加密程序中的密钥？目前有三种主流方法

3.1 密码学分析

白盒加密是一个密码学领域的知识，那么它必然也能被密码学的知识打败。

(https://www.qinless.com/wp-content/uploads/2022/04/6246feddc3737.png)

返回顶部

这要求分析者具有数学和密码学的知识储备，并且要具体分析样本中的白盒实现，这实在不是大多数人的长项（包括我）。

3.2 差分故障攻击（DFA）

DFA 的提出甚至早于白盒加密，它是灰盒攻击场景下的一个方案。Bos 等人惊喜的发现，用于对付标准加密实现的DFA 竟然也可以处理白盒加密。DFA 是目前最热门的方法之一，各种白盒加密破解赛中，都可见到它的身影。

稳定、好用、对工具依赖低是这个方案的优点。

3.3 差分计算攻击 （DCA）

DCA 的提出同样早于白盒加密，它本用于处理标准加密实现，过去它叫差分能量攻击（DPA），是灰盒攻击场景下的重要方案。同样是Bos 等人发现，在白盒攻击模型下稍微给它改一下，就能处理白盒加密。

因为，业界认为白盒加密目前的效果并不好，在设计上，它的强度本应能处理白盒威胁，但实际上，它却被更低一级的威胁，比如DCA/DFA 这样的灰盒攻击方法追着锤。

四、尾声

下一篇我们从DFA 开始说起。

标签： # aes，# dfa，# 攻击方法，# 攻击模型，# 白盒aes (https://www.qinless.com/tag/baiheaes)

(https://www.qinless.com/1645)

(https://www.qinless.com/wp-content/themes/honey/public/qrcode?data=https://www.qinless.com/1645)

(https://www.qinless.com/1645) 暂无评论 (https://www.qinless.com/1645#respond)

【转载】第一讲——从黑盒攻击模型到白盒攻击模型
上一篇
(https://www.qinless.com/1642)

【转载】第三讲——差分故障攻击的原理
下一篇
(https://www.qinless.com/1647)

会爬山的小脑虎 (https://www.qinless.com/author/xiayu)
小爬爬

返回顶部

本文作者： 会爬山的小脑虎 (https://www.qinless.com/author/xiayu)
本文链接： https://www.qinless.com/?p=1645 (https://www.qinless.com/1645)
版权声明：本博客所有文章除特别声明外，均采用 CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/) 许可协议。转载请注明来自 qinless 的博客！ (https://www.qinless.com)

微信搜一搜

逆向小友

发表评论

昵称 *

邮箱 *

网址

输入评论内容...

提交评论

返回顶部

Copyright 2022. All Rights Reserved. Powered By XinTheme (http://www.xintheme.com) + WordPress 果酱 (https://blog.wpjam.com/) 浙ICP备2021029793号 (http://www.miibeian.gov.cn/)

返回顶部